

Interface Capability Brief

CSV to JSON Demo · Stakeholder overview · 2026-08-05

1. Executive summary

Simple PilotFish demo that polls a folder for CSV files, converts them to JSON, and writes the JSON into an output folder.

2. Who this is for

- Sources: Operators / Web UI dropping .csv into input/inbound - Destinations: JSON files under output/json; archived CSV under output/archive - Demo vs production: Demo only

- Sources: Operators / Web UI dropping .csv into input/inbound
- Destinations: JSON files under output/json; archived CSV under output/archive
- Demo vs production: Demo only

3. What the interface can do

Capabilities below are drawn from the working design and, when present, the published CapabilityStatement — not from aspirational roadmap items.

- Transport: DirectoryListener polling \$\$CSV_INBOUND_DIRECTORY
- Format: comma-delimited CSV with a header row (Dialect A via CSV Transformation)
- Identity fields: source filename (com.pilotfish.FileName)
- Samples path: samples/*.csv

Integration highlights

Stage	Module / mechanism	Notes
Listener	com.pilotfish.eip.modules.file.DirectoryListener	.csv only; Move → archive
Source processor	com.pilotfish.eip.modules.transform.CSVTransformationProcessor	CSV → XML (Dialect A XCSDData)
FormatProfile	Relay	Pass-through
Router	com.pilotfish.eip.modules.routing.XPathRoutingModule	Always true() on XML → Write JSON
Target processor	com.pilotfish.eip.modules.transform.XSLTProcessor (csv-xml-to-json.xslt)	Dialect A XML → {"records":[...]} JSON text; after route so XPath router sees XML
Transport	com.pilotfish.eip.modules.file.DirectoryTransport	.json → \$\$JSON_OUTPUT_DIRECTORY

4. Honest boundaries (not claimed yet)

- Optional kickout directory for malformed CSV
- Optional XSLT polish for flatter { "records": [...] } JSON

5. What you can verify in a walkthrough

Use these as checklist items when reviewing the demo with business stakeholders. They describe outcomes, not module names.

- Exercise route “1 - Convert CSV To JSON” from the Web UI or documented smoke path
- Confirm outputs land in the documented folders / queues and appear in the Web UI status views

6. How it works (plain language)

7. Validation

- What is checked: file extension .csv only - What is NOT checked: schema, column presence, quote/escaping edge cases, JSON schema - Does failure block outbound? No dedicated kickout — bad CSV may fail the transaction (see logs/eip.log)

6. State & idempotency

- Status model: file presence only (no SQL) - When state advances: listener Moves CSV to archive after pick-up; JSON overwrite by basename - Dedup keys: none (demo overwrite) - Retry / poison: kickout path not implemented (documented)

8. Dual-write / side effects

- Order of commits: archive Move (listener) then JSON write - Compensation: none — accepted demo risk that archive can succeed while JSON write fails

3. Inbound contract

- Transport: DirectoryListener polling \$\$CSV_INBOUND_DIRECTORY - Format: comma-delimited CSV with a header row (Dialect A via CSV Transformation) - Identity fields: source filename (com.pilotfish.FileName) - Samples path: samples/*.csv

Routes in this interface

Each route is a configured PilotFish flow. Technical wiring appears in the Route Diagrams PDF; here is the stakeholder reading.

1 - Convert CSV To JSON: runs steps such as Poll CSV Files, CSV to XML, Conditional Router, XML to JSON.

7. Security & trust posture

- Credentials and passwords in this package are demo-only — not production secrets
- Risk noted in design: Med — Archive-before-JSON (listener Move) — Accepted demo risk; labeled in README
- Risk noted in design: Med — No schema validation / kickout — Explicitly not implemented
- Risk noted in design: Low — Native JSONTransformationProcessor emits duplicate sibling keys for Dialect A — Prefer XSLT {"records":[...]} for valid multi-row JSON
- Risk noted in design: Low — Docs/V2 may be newer than 23R1 — Modules present in modules-csv / modules-json 23R1 jars

8. Seeing it work

- - Ports: EIP 8108, Web UI 8109 (no SQL) - LAN: LAN_HINT=http://192.168.68.52:8109/ - Cold start ~60–90s for EIP
- docker compose up -d --build
- - Web UI local: http://localhost:8109/
- - Local PDF: http://localhost:8109/documents/route-diagrams.pdf

9. Related technical documents

-
- DESIGN.md — working engineering specification for this interface
 - README.md — how to run, ports, and smoke commands
 - CSV_to_JSON_V2_Route_Diagrams.pdf — detailed PilotFish route wiring diagrams
 - This Capability Brief — shareable stakeholder summary (regenerate after design changes)

This document is generated from the interface sources listed above. If design and runtime diverge, believe the smoke-tested runtime and update DESIGN.md.